Лекція №2

Введення у клієнт-серверні технології Веб

Протокол HTTP. Схема HTTP-сеансу. Склад HTTP-запиту. Безпека передачі даних HTTP. Cookie.

Базовим протоколом мережі гіпертекстових ресурсів Інтернет є протокол HTTP. В його основу покладено взаємодію "*клієнт-сервер*", тобто передбачається, що:

1. Споживач - *клієнт* ініціювавши з'єднання з постачальником-сервером, посилає йому запит;
2. Постачальник - *сервер*, отримавши запит, здійснює необхідні дії і повертає назад клієнту відповідь з результатом.

При цьому можливі два способи організації роботи комп'ютера-клієнта:

- *Тонкий клієнт* - це комп'ютер-клієнт, який переносить всі завдання обробки інформації на сервер. Прикладом тонкого клієнта може бути комп'ютер із браузером, який використовується для роботи з веб-застосунками.
- *Товстий клієнт*, навпаки, здійснює обробку інформації незалежно від сервера, використовує останній переважно лише зберігання даних.

Перш ніж перейти до конкретних клієнт-серверних веб-технологій, розглянемо основні принципи та структуру базового протоколу HTTP.

2.1 Протокол HTTP

HTTP (HyperText Transfer Protocol - RFC 1945, RFC 2616) - протокол прикладного рівня передачі гіпертексту.

Центральним об'єктом у HTTP є *ресурс,* який вказує URI у запиті клієнта. Зазвичай такими ресурсами є файли, що зберігаються на сервері. Особливістю протоколу HTTP є можливість вказати в запиті та відповіді

спосіб подання одного і того ж ресурсу за різними параметрами: форматом, кодуванням, мовою і т.д. призначений передачі символьної інформації. На перший погляд це може здатися зайвою витратою ресурсів. Дійсно, дані у символьному вигляді займають більше пам'яті, повідомлення створюють додаткове навантаження на канали зв'язку, однак такий формат має багато переваг. Повідомлення, що передаються по мережі, легкочитані, і, проаналізувавши отримані дані, системний адміністратор може легко знайти помилку та усунути її. При необхідності роль одного з програм, що взаємодіють, може виконувати людина, вручну вводячи повідомлення в потрібному форматі.

На відміну від інших протоколів, HTTP є протоколом без пам'яті. Це означає, що протокол не зберігає інформацію про попередні запити клієнтів та відповіді сервера. Компоненти, що використовують HTTP, можуть самостійно здійснювати збереження інформації про стан, пов'язаний з останніми запитами та відповідями. Наприклад, клієнтська веб-програма, яка надсилає запити, може відстежувати затримки відповідей, а веб-сервер може зберігати IP-адреси та заголовки запитів останніх клієнтів.

Все програмне забезпечення для роботи з протоколом HTTP поділяється на три основні категорії:

- *Сервери* – постачальники послуг зберігання та обробки інформації (обробка запитів).
- *Клієнти* – кінцеві споживачі послуг сервера (надсилання запитів).
- *Проксі-сервери* для підтримки транспортних служб.

Основними клієнтами є *браузери,* наприклад: Google Chrome, Microsoft Edge, Mozilla Firefox, Opera, Safari, Brave, Vivaldi, Tor Browser та інші. Найбільш популярними реалізаціями веб-серверів є Internet Information Services (IIS), Apache, lighttpd, nginx. Найбільш відомі реалізації проксі-серверів: Squid, Nginx, HAProxy, Traefik, Apache Traffic Server, 3proxy, Shadowsocks, SOCKS5 Proxy, Privoxy.

"Класична" схема HTTP-сеансу виглядає так.

1. Встановлення TCP-з'єднання.
2. Запит клієнта.
3. Відповідь сервера.
4. Розрив TCP-з'єднання.

Таким чином, клієнт надсилає серверу запит, отримує від нього відповідь, після чого взаємодія припиняється. Зазвичай запит клієнта є вимогою передати HTML-документ або якийсь інший ресурс, а відповідь сервера містить код цього ресурсу.

До складу HTTP-запиту, що передається клієнтом серверу, входять такі компоненти.

- Рядок стану (іноді для її позначення використовують терміни рядок-статус, або рядок запиту).
- Поля заголовка.
- Порожня стрічка.
- Тіло запиту.

Рядок стану разом з *полями заголовка* іноді називають також *заголовком запиту.*

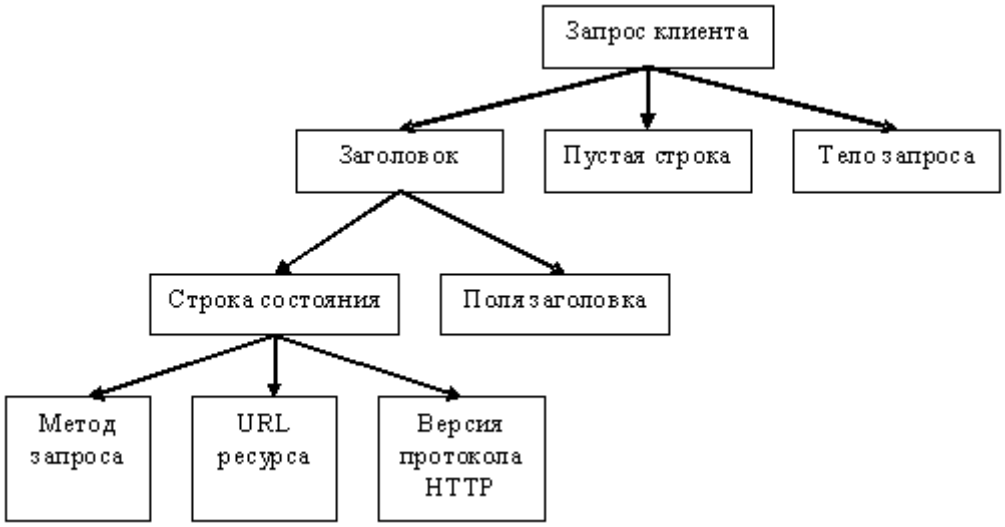

Рисунок 2.1 – Структура запиту клієнта.

Рядок стану має наступний формат:

метод_запиту URL_ресурса версія_протоколу_HTTP

Розглянемо компоненти рядка стану, у своїй особливу увагу приділимо методам запиту.

Метод, вказаний у рядку стану, визначає спосіб на ресурс, URL якого заданий у тому рядку. Метод може приймати значення GET, POST, HEAD, PUT, DELETE, PATCH, OPTIONS, CONNECT, TRACE. Незважаючи на безліч методів, для веб-програміста по-справжньому важливі лише два з них: GET і POST.

- GET. Відповідно до формального визначення, метод GET призначається для отримання ресурсу із вказаною URL-адресою. Отримавши запит GET, сервер повинен прочитати вказаний ресурс і включити код ресурсу до відповіді клієнту. Ресурс, URL якого передається у складі запиту, не обов'язково повинен бути HTML-сторінкою, файлом із зображенням або іншими даними. URL ресурсу може вказувати на виконуваний код програми, який за дотримання певних умов повинен бути запущений на сервері. В цьому випадку клієнту повертається не код програми, а дані, що згенеровані в процесі її виконання. Незважаючи на те, що, за визначенням, метод GET призначений для отримання інформації, він може застосовуватися і в інших цілях. Метод GET цілком підходить передачі невеликих фрагментів даних на сервер.
- POST. Згідно з тим самим формальним визначенням, основне призначення методу POST - передача даних на сервер. Однак, подібно до методу GET, метод POST може застосовуватися по-різному і нерідко використовується для отримання інформації з сервера. Як і у випадку з методом GET URL, заданий у рядку стану, вказує на конкретний ресурс. Метод POST також можна використовувати для запуску процесу.

Ідемпотентність — це властивість операції, яка гарантує, що незалежно від кількості повторних викликів результат залишиться незмінним.

У HTTP до ідемпотентних методів належать:

GET — отримання ресурсу не змінює його стан

HEAD — як GET, але повертає лише заголовки

PUT — створює або оновлює ресурс, кожен виклик з тими ж даними не змінює результат

DELETE — видаляє ресурс, повторний виклик не призведе до іншого ефекту (якщо ресурс уже видалено)

OPTIONS — повертає допустимі методи для ресурсу без його зміни

POST не є ідемпотентним, бо кожен запит може створювати нові ресурси або змінювати стан сервера по-різному.

- Методи HEAD та PUT є модифікаціями методів GET та POST.

Версія протоколу HTTP зазвичай задається в наступному форматі:

HTTP/версія.модифікація

Поля заголовка, які йдуть за рядком стану, дозволяють уточнювати запит, тобто. надсилати серверу додаткову інформацію. Поле заголовка має такий формат:

Ім'я_поля: Значення

Призначення поля визначається його ім'ям, яке відокремлюється від значення двокрапкою.

Імена деяких полів заголовка, що найчастіше зустрічаються в запиті клієнта, і їх призначення наведені в таблиці 2.1 .

Таблиця 2.1. Поля заголовка запиту HTTP.

Поля заголовка HTTP-запиту	Значення
Host	Доменне ім'я або IP-адреса вузла, до якого звертається клієнт

Referer	URL документа, який посилається на ресурс, вказаний у рядку стану
від	Адреса електронної пошти користувача, який працює з клієнтом
Accept	MIME-типи даних, що обробляються клієнтом. Це поле може мати кілька значень, що відокремлюються одне від одного комами. Часто поле заголовка Accept використовується для того, щоб повідомити сервер про те, які типи графічних файлів підтримує клієнт
Accept-Language	Набір двохсимвольних ідентифікаторів, розділених комами, які позначають мови, які клієнт підтримує
Accept-Charset	Перелік наборів символів, що підтримуються
Content-Type	MIME-тип даних, які у тілі запиту (якщо запит складається з одного заголовка)
Content-Length	Число символів, що містяться в тілі запиту (якщо запит не складається з одного заголовка)
Range	Є в тому випадку, якщо клієнт запитує не весь документ, а лише його частина
Connection	Використовується для керування TCP-з'єднанням. Якщо поле містить Close, це означає, що після обробки запиту сервер повинен закрити з'єднання. Keep-Alive пропонує не закривати TCP-з'єднання, щоб воно могло бути використане для наступних запитів
User-Agent	Інформація про клієнта

У багатьох випадках під час роботи в Інтернеті тіло запиту відсутнє. При запуску CGI-сценаріїв дані, що передаються для них у запиті, можуть бути розміщені в тілі запиту.

Нижче наведено приклад HTML-запиту, згенерованого браузером

GET http://oak.oakland.edu/ HTTP/1.0

Connection: Keep-Alive

User-Agent: Mozilla/4.04 [en] (Win95; I)

Host: oak.oakland.edu

Accept: image/gif, image/x-xbitmap, image/jpeg, image/pjpeg, image/png, */*

Accept-Language: en

Accept-Charset: iso-8859-l,*,utf-8

Отримавши від клієнта запит, сервер має відповісти йому. Знання структури відповіді сервера необхідне розробнику веб-додатків, оскільки програми, які виконуються сервері, повинні самостійно формувати відповідь клієнту.

Подібно до запиту клієнта, відповідь сервера також складається з чотирьох наведених нижче компонентів.

- Рядок стану.
- Поля заголовка.
- Порожня стрічка.
- Тіло відповіді.

Відповідь сервера клієнту починається з рядка стану, який має наступний формат:

Версія_протоколу Код_відповіді Пояснення_повідомлення

- **Версия_протокола** задається у тому форматі, що у запиті клієнта, і має той самий сенс.
- **Код_відповіді** - це тризначне десяткове число, яке представляє в закодованому вигляді результат обслуговування запиту сервером.
- **Пояснювальне повідомлення** дублює код відповіді в символьному вигляді. Це рядок символів, який не обробляється клієнтом. Вона призначена для системного адміністратора або оператора, що займається обслуговуванням системи, та є розшифровкою коду відповіді.

З трьох цифр, що становлять код відповіді, перша (старша) визначає клас відповіді, решта двох є номером відповіді всередині класу. Так, наприклад, якщо запит був успішно оброблений, клієнт отримує наступне повідомлення:

HTTP/1.0 200 OK

Як видно, за версією протоколу HTTP 1.0 слідує код 200. У цьому коді символ 2 означає успішну обробку запиту клієнта, а дві цифри (00) — номер даного повідомлення.

У реалізаціях протоколу HTTP, що використовуються в даний час, перша цифра не може бути більше 5 і визначає наступні класи відповідей.

- 1 – спеціальний клас повідомлень, званих інформаційними. Код відповіді, який починається з 1, означає, що сервер продовжує обробку запиту. При обміні даними між HTTP-клієнтом та HTTP-сервером повідомлення цього класу використовуються досить рідко.
- 2 – успішна обробка запиту клієнта.
- 3 - перенаправлення запиту. Щоб запит був обслужений, необхідно вжити додаткових дій.
- 4 – помилка клієнта. Як правило, код відповіді, що починається з цифри 4, повертається в тому випадку, якщо запит клієнта зустріла синтаксичну помилку.
- 5 – помилка сервера. З тих чи інших причин сервер не може виконати запит.

Приклади кодів відповідей, які клієнт може отримати від сервера, та пояснювальні повідомлення наведено у таблиці 2.2 .

Таблиця 2.2. Класи кодів відповіді сервера

Код	Розшифровка	Інтерпретація
100	Continue	Частину запиту прийнято, і сервер чекає від клієнта продовження запиту
200	OK	Запит успішно оброблений, і у відповіді клієнта передаються дані, вказані у запиті

201	Created	В результаті обробки запиту було створено новий ресурс
202	Accepted	Запит прийнятий сервером, але його обробка не закінчена. Цей код відповіді не гарантує, що запит буде опрацьовано без помилок.
206	Partial Content	Сервер повертає частину ресурсу у відповідь на запит, що містив поле заголовка Range
300	Multiple Choice	Запит вказує більш ніж один ресурс. У тілі відповіді можуть бути вказівки на те, як правильно ідентифікувати запитуваний ресурс
301	Moved Permanently	Затребуваний ресурс більше не розміщується на сервері
302	Moved Temporarily	Затребуваний ресурс тимчасово змінив свою адресу
400	Bad Request	У запиті клієнта виявлено синтаксичну помилку
403	Forbidden	Наявний на сервері ресурс недоступний для цього користувача
404	Not Found	Ресурс, вказаний клієнтом, на сервері відсутній
405	Method Not Allowed	Сервер не підтримує метод, вказаний у запиті
500	Internal Server Error	Один із компонентів сервера працює некоректно
501	Not Implemented	Функціональних можливостей сервера недостатньо, щоб виконати запит клієнта
503	Service Unavailable	Служба тимчасово недоступна

505	HTTP Version no Supported	Версія HTTP, вказана у запиті, не підтримується сервером

У відповіді використовується така сама структура полів заголовка, як і запит клієнта. Поля заголовка призначені у тому, щоб уточнити відповідь сервера клієнту. Опис деяких полів, які можна зустріти в заголовку відповіді сервера, наведено в таблиці 2.3 .

Таблиця 2.3. Поля заголовка відповіді веб-сервера.

Ім'я поля	Опис вмісту
Server	Ім'я та номер версії сервера
Age	Час у секундах, що минув з моменту створення ресурсу
Allow	Список методів, допустимих для цього ресурсу
Content-Language	Мови, які повинен підтримувати клієнт для того, щоб коректно відобразити ресурс, що передається
Content-Type	MIME-тип даних, які у тілі відповіді сервера
Content-Length	Число символів, що містяться у тілі відповіді сервера
Last-Modified	Дата та час останньої зміни ресурсу
Date	Дата та час, що визначають момент генерації відповіді
Expires	Дата і час, що визначають момент, після якого інформація, передана клієнту, вважається застарілою
Location	У цьому полі вказується реальне розташування ресурсу. Воно використовується для перенаправлення запиту
Cache-Control	Директиви керування кешуванням. Наприклад, no - cache означає, що дані не повинні кешуватися

У тілі відповіді міститься код ресурсу, що передається клієнту у відповідь на запит. Це не обов'язково має бути HTML-текст веб-сторінки. У складі відповіді можуть передаватися зображення, аудіофайл, фрагмент відеоінформації, а також будь-який інший тип даних, що підтримуються

клієнтом. Про те, як слід обробляти отриманий ресурс, клієнт повідомляє вміст поля заголовка Content-Type.

Нижче наведено приклад відповіді сервера на запит, наведений у попередньому розділі. Тіло відповіді містить вихідний текст HTML-документа.

```
HTTP/1.1 200 OK
Server: Microsoft-IIS/5.1
X-Powered-By: ASP.NET
Date: Mon, 20 OCT 2008 11:25:56 GMT
Content-Type: text/html
Accept-Ranges: bytes
Last-Modified: Sat, 18 Oct 2008 15:05:44 GMT
ETag: "b66a667f948c92:8a5"
Content-Length: 426

<html>
<body>
<form action='http://localhost/Scripts/test.pl'>
<p>Operand1: <input type='text' name='A'></p>
<p>Operand2: <input type='text' name='B'></p>
<p>Operation:<br>
<select name='op'>
<option value='+'>+</option>
<option value='-'>-</option>
<option value='*'>*</option>
<option value='/'>/</option>
<select></p>
<input type='submit' value='Calculate!'>
</from>
</body>
```

</html>

Поля заголовка та тіло повідомлення можуть бути відсутніми, але рядок стану є обов'язковим елементом, оскільки вказує на тип запиту/відповіді.

Поле з ім'ям Content-type може зустрічатися як у запиті клієнта, так і у відповіді сервера. Як значення цього поля вказується MIME -тип вмісту запиту чи відповіді. MIME -тип також передається в поле заголовка Accept, що є у запиті.

Специфікація MIME (Multipurpose Internet Mail Extension – багатоцільове поштове розширення Internet) спочатку була розроблена для того, щоб забезпечити передачу різних форматів даних у складі електронних листів. Однак, використання MIME не вичерпується електронною поштою. Кошти MIME успішно використовуються у WWW і, по суті, стали невід'ємною частиною цієї системи.

Стандарт MIME розроблений як специфікація, що розширюється, в якій мається на увазі, що число типів даних буде зростати в міру розвитку форм подання даних. Кожен новий тип обов'язково повинен бути зареєстрований в IANA (Internet Assigned Numbers Authority).

До появи MIME комп'ютери, що взаємодіють за протоколом HTTP, обмінювалися виключно текстовою інформацією. Для передачі зображень, як і передачі будь-яких інших двійкових файлів, доводилося користуватися протоколом FTP.

Відповідно до специфікації MIME для опису формату даних використовуються *тип* і *підтип. Тип* визначає, до якого класу відноситься формат вмісту запиту HTTP або відповіді HTTP. *Підтип* уточнює формат. Тип і підтип відокремлюються один від одного косою межею:

тип/підтип

Оскільки у переважній більшості випадків у відповідь на запит клієнта сервер повертає вихідний текст HTML-документа, то в полі Content-type відповіді зазвичай міститься значення text/html. Тут ідентифікатор text визначає тип, повідомляючи, що клієнту передається символьна інформація, а

ідентифікатор html визначає підтип, тобто. вказує на те, що послідовність символів, що міститься в тілі відповіді, є описом документа мовою HTML.

Список типів і підтипів MIME досить великий. У таблиці 2.4 наведено приклади MIME -типів, що найчастіше зустрічаються в заголовках HTML-запитів та відповідей.

Таблиця 2.4. MIME типи даних.

Тип/підтип	Розширення	Опис
application/pdf	.pdf	Документ для обробки Acrobat Reader
application/msexcel	.xls	Документ у форматі Microsoft Excel
application/postscript	.ps,.eps	Документ у форматі PostScript
application/x-tex	.tex	Документ у форматі TIX
application/msword	.doc	Документ у форматі Microsoft Word
application/rtf	.rtf	Документ у форматі RTF, який відображається за допомогою Microsoft Word
image/gif	.gif	Зображення у форматі GIF
image/jpeg	.jpeg,.jpg,.	Зображення у форматі JPEG
image/tiff	.tiff,.tif	Зображення у форматі TIFF
image/webp	.webp	Зображення у форматі WebP
image/avif	.avif	Зображення у форматі AVIF
image/x-xbitmap	.xbm	Зображення у форматі XBitmap
text/plain	.txt	ASCII-текст
text/html	. html,. htm	Документ у форматі HTML
audio/midi	.midi,.mid	Аудіофайл у форматі MIDI
audio/x-wav	.wav	Аудіофайл у форматі WAV
audio/mp3	.mp3	Аудіофайл у форматі MP3
message/rfc822		Поштове повідомлення

message/news		Повідомлення у групи новин
video/mpeg	.mpeg,.mpg,.mpe	Відеофрагмент у форматі MPEG
video/avi	.avi	Відеофрагмент у форматі AVI

Типи MIME, також відомі як типи Інтернет-медіа, визначають природу та формат документа, файлу або набору байтів. Вони відіграють ключову роль у забезпеченні правильної обробки даних в інтернеті, вказуючи, як файл має бути використаний або виконаний. Типи MIME стандартизовані Інтернет-асоціацією з присвоєння номерів (IANA) та можуть бути знайдені в RFC 6838 IETF. Вони важливі для того, щоб веб-сервери надсилали правильний тип MIME у заголовку відповіді Content-Type, дозволяючи браузерам правильно обробляти URL. Неправильна настройка типу MIME може призвести до неправильної інтерпретації вмісту файлів браузерами, що, у свою чергу, призведе до некоректної роботи сайтів або неправильної обробки завантажених файлів.

Існують різні категорії типів MIME, включаючи:

- Дискретні типи, такі як `application`, `audio`, `image`, `text`, `video` і `font`, кожен з конкретними підтипами для різних видів даних. Наприклад, `application/json` для даних JSON, `audio/mpeg` для аудіо у форматі MP3, `image/jpeg` для зображень JPEG, `text/html` для документів HTML і `video/mp4` для відео у форматі MP4.

- Багаточастинні типи використовуються для документів, що складаються з декількох частин (кожна з яких може мати свій власний тип MIME) або для інкапсуляції декількох файлів, що надсилаються разом у одній транзакції, таких як електронні листи з декількома вкладеннями. Приклади включають `multipart/form-data` для надсилання форм, що включають завантаження файлів, і `multipart/mixed` для електронних листів.

multipart/form-data — це один із значень атрибута enctype у HTML-формах, який використовується для передачі файлів та інших даних, що містять бінарний контент. Його потрібно вказувати у формі, якщо вона містить <input type="file">, інакше браузер не зможе правильно відправити файли.

Експериментальні та специфічні для виробника типи як, наприклад, `application/vnd.google-earth.kml+xml` для файлів KML (використовуваних Google Earth) і `application/vnd.ms-excel` для файлів Microsoft Excel. Ці типи дозволяють передавати власні формати даних через інтернет.

Крім того, у офіційному реєстрі IANA представлені сучасні та нещодавно зареєстровані типи MIME, що відображають розвиток інтернет-технологій та форматів даних. Приклади включають `application/cbor` для даних CBOR, `application/cbor-seq` для послідовностей CBOR та багато інших, адаптованих до конкретних застосувань та технологій. Типи MIME є важливим аспектом інтернет-комунікацій, забезпечуючи точну інтерпретацію та обробку даних на різних платформах та застосунках. Для отримання повного переліку типів MIME та їх визначень найбільш авторитетним джерелом є реєстр Media Types IANA.

Для однозначної ідентифікації ресурсів мережі Веб використовуються унікальні ідентифікатори URL.

Єдиний ідентифікатор ресурсу URI (Uniform Resource Identifier) є короткою послідовністю символів, що ідентифікує абстрактний або фізичний ресурс. URI не вказує на те, як отримати ресурс, а лише його ідентифікує. Це дозволяє описувати за допомогою RDF (Resource Description Framework) ресурси, які не можуть бути отримані через Інтернет (імена, назви тощо). Найвідоміші приклади URI - це URL та URN.

- URL (Uniform Resource Locator) - це URI, який крім ідентифікації ресурсу, надає ще й інформацію про місцезнаходження цього ресурсу.

- URN (Uniform Resource Name) - це URI, який ідентифікує ресурс у певному просторі імен, але на відміну від URL, URN не вказує на місцезнаходження цього ресурсу.

URL має таку структуру:

<схема>://<логін>:<пароль>@<хост>:<порт>/<URL-шлях>

де:

- схема – схема звернення до ресурсу (зазвичай мережевий протокол);
- логін - ім'я користувача, яке використовується для доступу до ресурсу;
- пароль – пароль, асоційований із зазначеним ім'ям користувача;
- хост - повністю прописане доменне ім'я хоста в системі DNS або IP-адреса хоста;
- порт – порт хоста для підключення;
- URL-шлях - уточнююча інформація про місце знаходження ресурсу.

Загальноприйняті схеми (протоколи) URL включають протоколи: *ftp, http, https, telnet,* а також:

- gopher - протокол Gopher;
- mailto - адреса електронної пошти;
- news — новини Usenet;
- nntp - новини Usenet через протокол NNTP;
- irc - протокол IRC;
- prospero - служба каталогів Prospero Directory Service;
- wais - база даних системи WAIS;
- xmpp - протокол XMPP (частина Jabber);
- file – ім'я локального файлу;
- data - безпосередні дані (Data: URL);

Зараз багато старих протоколів замінені більш безпечними або ефективними аналогами.

FTP → SFTP, FTPS (SFTP працює поверх SSH, FTPS додає шифрування до FTP)

HTTP → HTTPS (HTTP без шифрування майже не використовується)

Telnet → SSH (Telnet передає все у відкритому вигляді, SSH шифрує)

SMTP, IMAP, POP3 → їхні захищені версії SMTPS, IMAPS, POP3S

Ще багато компаній відмовляються від FTP/SFTP на користь S3, WebDAV, rsync, Syncthing або хмарних сервісів типу Google Drive, OneDrive, Dropbox.

2.2 Безпека передачі даних HTTP

Оскільки протокол HTTP призначений для передачі символьних даних у відкритому (незашифрованому) вигляді, особи, які мають доступ до каналу передачі даних між клієнтом і сервером, можуть легко переглядати весь трафік і використовувати його для здійснення несанкціонованих дій. У зв'язку з цим запропоновано низку розширень базового протоколу, спрямованих на підвищення захищеності інтернет-трафіку від несанкціонованого доступу.

Найпростішим є розширення HTTPS, при якому дані, що передаються за протоколом HTTP, "упаковуються" в криптографічний протокол SSL або TLS, забезпечуючи тим самим захист цих даних. На відміну від HTTP який має 80 порт, для HTTPS за замовчуванням використовується TCP-порт 443. Щоб підготувати веб-сервер для обробки HTTPS з'єднань, адміністратор повинен отримати та встановити сертифікат для цього веб-сервера.

SSL (Secure Sockets Layer) - криптографічний протокол, що забезпечує безпечну передачу даних через Інтернет. Під час його використання створюється захищене з'єднання між клієнтом та сервером. SSL спочатку розроблено компанією Netscape Communications. Згодом на підставі протоколу SSL 3.0 було розроблено та прийнято стандарт RFC, який отримав назву TLS. Цей протокол використовує шифрування з відкритим ключем для підтвердження справжності передавача та одержувача. Підтримує надійність передачі даних за рахунок використання коригувальних кодів та безпечних хеш-функцій. На нижньому рівні багаторівневого транспортного протоколу (наприклад TCP) він є протоколом запису і використовується для інкапсуляції різних протоколів (POP3, IMAP, SMTP або HTTP). Для кожного інкапсульованого протоколу він забезпечує умови, за яких сервер і клієнт можуть підтверджувати один одному свою справжність, виконувати алгоритми шифрування і обмін криптографічними ключами, перш ніж протокол прикладної програми почне передавати і отримувати дані.

Для доступу до веб-сторінок, захищених протоколом SSL, в URL замість схеми http, як правило, підставляється схема https, яка вказує на те, що використовуватиметься SSL-з'єднання. Стандартний TCP-порт для з'єднання за протоколом https - 443. Для роботи SSL потрібно, щоб на сервері був SSL-*сертифікат.*

Let's Encrypt – це безкоштовний центр сертифікації (CA), який видає SSL/TLS-сертифікати. Його використовують для шифрування трафіку на вебсайтах.

У мережі Інтернет підтримуються 3 типи аутентифікації при клієнт-серверних взаємодіях:

- Basic - базова автентифікація, коли ім'я користувача і пароль передаються в заголовках http-пакетів. Пароль при цьому не шифрується і присутній у чистому вигляді кодування base64. Для цього аутентифікації використання SSL є обов'язковим.

encodedString = toBase64(login + ":" + password)

Authorization: Basic {encodedString}

Base64 — це метод кодування бінарних даних у текстовий формат, що складається з 64 символів:

латинські A-Z, a-z, цифри 0-9, плюс + і слеш /. Також використовується = як заповнювач, якщо довжина не кратна 3.

Декодування повертає вихідний бінарний формат.

Декодування повертає вихідний бінарний формат, тому Base64 не є шифруванням, а лише способом кодування. Це робить його небезпечним для:

Зберігання паролів — Base64 можна легко розкодувати, тому його не можна використовувати для захисту паролів. Замість цього варто використовувати хешування (наприклад, bcrypt, Argon2).

Передачі конфіденційних даних — якщо передати Base64-кодований API-ключ або токен, його можна просто декодувати. Краще використовувати шифрування (AES, TLS).

Захисту файлів — файли, закодовані в Base64, не отримують жодного захисту. Це тільки зручний формат для передачі, але не для безпеки.

Обфускації коду — іноді код кодують в Base64, думаючи, що це захист, але насправді це легко розкодується (atob() у браузері або base64 -d в терміналі).

Тому Base64 підходить тільки для зручності зберігання та передачі даних, а не для захисту.

- Digest - дайджест-автентифікація, коли пароль користувача передається в хешированном вигляді. За рівнем конфіденційності паролів цей

тип мало чим відрізняється від попереднього, тому що атакуючому все одно, чи справді це справжній пароль або тільки хеш від нього: перехопивши посвідчення, він все одно отримує доступ до кінцевої точки. Для цього аутентифікації використання SSL є обов'язковим.

- Integrated - інтегрована автентифікація, коли клієнт і сервер обмінюються повідомленнями для з'ясування справжності одне одного з допомогою протоколів NTLM чи Kerberos. Цей тип автентифікації захищений від перехоплення посвідчень користувачів, тому протокол SSL не потрібний для нього. Тільки при використанні даного типу аутентифікації можна працювати за схемою http, у всіх інших випадках необхідно використовувати https.

2.3 Cookie

Оскільки HTTP-сервер не пам'ятає передісторії запитів клієнтів, то кожен запит обробляється незалежно від інших, і сервер не має можливості визначити, чи виходять запити від одного клієнта або різних клієнтів.

Якщо сервер перевірятиме TCP-з'єднання і запам'ятовуватиме IP-адреси комп'ютерів-клієнтів, він все одно не зможе розрізнити запити від двох браузерів, що виконуються на одній машині. І навіть якщо припустити, що на комп'ютері працює лише одна клієнт-програма, ніхто не може стверджувати, що в проміжку між двома запитами вона не була завершена, а потім запущена знову вже іншим користувачем.

Тим не менш, якщо ви коли-небудь користувалися поштовою скринькою на gmail.com або на іншому сервері, що надає поштові послуги користувачам Веб, згадайте, як поводився клієнт після того, як ви створили для себе поштову скриньку на сервері. Коли ви наступного разу звернулися з того ж комп'ютера до gmail.com, ви, ймовірно, помітили, що після завантаження веб-сторінки ваше реєстраційне ім'я вже відображалося у відповідному полі введення.

Така інформація дозволяє отримати додатковий засіб під назвою cookie. Механізм cookie дозволяє серверу зберігати інформацію на комп'ютері клієнта та витягувати її звідти.

Ініціатором запису cookie є сервер. Якщо у відповіді сервера є поле заголовка Set-cookie, клієнт сприймає це як команду на запис cookie. Надалі, якщо клієнт звертається до сервера, від якого він раніше прийняв поле заголовка Set-cookie, крім іншої інформації він передає серверу дані cookie. Для передачі вказаної інформації серверу використовується поле заголовка cookie.

Щоб узагальнено уявити, як відбувається обмін даними cookie, розглянемо наступний приклад. Припустимо, що клієнт передає запити на сервери **A, B** та **C.** Припустимо також, що сервер, на відміну від **A** і **C,** передає клієнту команду записати cookie. Послідовність запитів клієнта серверу та відповіді на них виглядатиме приблизно таким чином.

1. Надсилання запиту серверу **A.**
2. Отримання відповіді сервера **A.**
3. Надсилання запиту серверу **B.**
4. Отримання відповіді сервера **B.** До складу відповіді входить поле заголовка SetCookie. Отримавши його, клієнт записує cookie на диск.
5. Надсилання запиту серверу **C.** Незважаючи на те, що на диску зберігається запис cookie, клієнт не робить жодних спеціальних дій, оскільки значення cookie було записано з ініціативи іншого сервера.
6. Отримання відповіді сервера **C.**
7. Надсилання запиту серверу **A.** У цьому випадку клієнт також не реагує на той факт, що на диску зберігається cookie.
8. Отримання відповіді сервера **A.**
9. Надсилання запиту серверу **B.** Перед тим, як сформувати запит, клієнт визначає, що на диску зберігається запис cookie, створений після отримання відповіді від **сервера.** Клієнт перевіряє, чи задовольняє цей запит деяким вимогам, і, якщо перевірка дає позитивний результат, включає в заголовок запиту поле Cookie.

Таким чином, процедуру запису та отримання cookie можна уявити як своєрідний "запит" сервера, інкапсульований у його відповіді клієнту.

Відповідно отримання cookie також можна уявити як відповідь клієнта, інкапсульований у складі запиту тому серверу.

Розглянемо докладніше, які дані передаються у полі заголовка Set-cookie та як вони впливають на поведінку клієнта.

Поле Set-cookie має такий формат:

Set-cookie: ім'я = значення; expires = дата; path = шлях; domain = ім'я_домену, secure

де

- Пара ім'я = значення – іменовані дані, які зберігаються за допомогою механізму cookie. Ці дані повинні зберігатися на клієнт-машині та передаватися серверу у складі чергового запиту клієнта.
- Дата, яка є значенням параметра expires, визначає час, після якого інформація cookie втрачає свою актуальність. Якщо ключового слова expires немає, дані cooki е видаляються після закінчення поточного сеансу роботи браузера.
- Значення параметра domain визначає домен, з яким зв'язуються дані cookie. Щоб дізнатися, чи слід передавати у складі запиту дані cookie, браузер порівнює доменне ім'я сервера, до якого він збирається звернутися, з доменами, пов'язаними із записами cookie, що зберігаються на клієнт-машині. Результат перевірки буде вважатися позитивним, якщо сервер, якому надсилається запит, належить домену, пов'язаному з cookie. Якщо відповідність не виявлена, дані cookie не надсилаються.
- Шлях, вказаний як значення параметра path, дозволяє виконати подальшу перевірку та прийняти остаточне рішення про те, чи слід передавати дані cookie у складі запиту. Крім домену із записом cookie зв'язується шлях. Якщо браузер виявив відповідність імені домену значенню параметра domain, він перевіряє, чи шлях до ресурсу шляху, пов'язаного з cookie. Порівняння вважається успішним, якщо ресурс міститься в каталозі, вказаному за допомогою ключового слова path або в одному з його підкаталогів. Якщо ця перевірка дає позитивний результат, дані cookie передаються серверу. Якщо

параметр path у полі Set-Cookie відсутній, то вважається, що запис cookie пов'язаний з URL-адресою конкретного ресурсу, що передається сервером клієнту.

- Останній параметр, secure, вказує на те, що дані cookie повинні передаватися захищеним каналом.

Для передачі даних cookie серверу використовується поле заголовка cookie. Формат цього поля досить простий:

Cookie: ім'я = значення; ім'я = значення; ...

За допомогою поля Cookie передається одна або кілька пар ім'я = значення. Кожна з цих пар належить запису cookie, для якого URL запитуваного ресурсу відповідають імені домену та шляху, зазначеним раніше в поле Set-cookie.

Cookie та законодавство – це окрема тема, оскільки використання файлів cookie регулюється різними законами про конфіденційність. Основні вимоги залежать від регіону.

Європа (GDPR, ePrivacy Directive)

У країнах ЄС діють суворі правила щодо файлів cookie. По-перше, якщо сайт використовує необов’язкові cookie (аналітика, реклама, персоналізація), то потрібна явна згода користувача. По-друге, сайт має пояснювати, які cookie використовуються, навіщо вони потрібні і як їх можна вимкнути.

Також важливим є право на відмову – користувач має змогу так само легко відмовитися від небазових cookie, як і погодитися на їх використання. Це означає, що простий банер «Продовжуючи використовувати сайт, ви погоджуєтесь…» більше не підходить.

Що потрібно зробити?

✅ Показувати банер про cookie при першому відвідуванні.

✅ Дозволяти змінювати налаштування cookie.

✅ Не зберігати аналітичні або рекламні cookie до отримання згоди.

США (CCPA, CPRA)

У США закони менш жорсткі, ніж у ЄС, але все одно є важливі вимоги. Користувачі мають право дізнатися, які дані збираються через cookie, а також відмовитися від їх використання для передачі третім сторонам (наприклад, рекламним платформам Google або Meta).

Згода на використання cookie не завжди обов’язкова, але сайт зобов’язаний чітко пояснювати свою політику збору даних. У деяких випадках користувач повинен мати можливість натиснути кнопку "Не продавати мої персональні дані".

Україна

В Україні законодавство про cookie поки що не таке жорстке, як у ЄС, і в основному орієнтується на Закон України "Про захист персональних даних". Якщо файли cookie містять персональні дані (наприклад, зберігають email або унікальний ідентифікатор), їх обробка регулюється так само, як і будь-яка інша персональна інформація.

Що важливо пам'ятати?

Функціональні cookie (сесійні, для входу) можна використовувати без згоди.

Аналітичні та рекламні cookie часто вимагають згоди.

Штрафи за порушення можуть бути значними, особливо в ЄС.

Якщо сайт працює на міжнародну аудиторію, краще одразу реалізувати cookie-банер із налаштуваннями та політику конфіденційності, щоб уникнути юридичних ризиків.